SAXESS™

Exchange Message Protocol (XMP) - Specification

v 40.10.34.1, 2007-03-07 (2007-04-11 10:59)

This document describes the Exchange Message Protocol (XMP) that provides services on the session and presentation layer (OSI layers 5 and 6).

Copyright Notice

Copyright © 2006 OMX Technology AB

All rights reserved.

SAXESS™ is a trademark of OMX AB.

Disclaimer and Limitation of Liability

Whilst all reasonable care has been taken to ensure that the details are true and not misleading at the time of publication, no liability whatsoever is assumed by OMX AB, or any subsidiary of OMX AB, with respect to the accuracy or any use of the information provided herein.

Any license, delivery and support of software systems etc. require entering into separate agreements with OMX Technology AB.

This document contains confidential information and may not be modified or reproduced, in whole or in part, or transmitted in any form to any third party, without the written approval from OMX Technology AB.

Contents

1. Introduction
1.1. Purpose
1.2. Scope
2. Overview
3. XMP Header Details
3.1. XMP Base Header
3.1.1. XMP Body Types
3.2. XMP Extension Header
3.2.1. Extension Header Entry Tags
3.2.1.1. Compression Algorithms
3.2.1.2. Encryption Algorithms
3.2.1.3. Session States
3.2.1.4. Protocol Versions
3.2.1.5. Extension Header Entry Examples
3.2.2. Extension Header Examples
3.2.2.1. Round Trip Instrumentation
3.2.2.2. Interactive KeepAlive with Data
3.2.2.3. Interactive Dataless KeepAlive
3.2.2.4. Unicast / Multicast KeepAlive
3.2.2.5. QueueAlert Configuration Request
3.2.3. Timestamp Request
4. Encapsulation
4.1. Encapsulation Examples
4.1.1. Example - An Encrypted XTP message
4.1.2. Example - A Compressed Sequence of Frames
5. Key Exchange (for Session Encryption)

1. Introduction

XMP is a protocol that provides typed messages, an optional "Extension Header" and a facility for message encapsulation. The encapsulation mechanism provides support for compression and encryption.

The SAXESS system and its clients interact using XTP messages that are encapsulated in XMP frames. The XMP frames are sent over a TCP/IP session. The table below shows how the various protocols used can be located in the OSI reference model.

TABLE 1: Mapping to the OSI reference model

Acronym	OSI	Full Name
XTP	7 Application	Exchange Transaction Protocol
XMP	6 Presentation 5 Session	Exchange Message Protocol
TCP	4 Transport	Transmission Control Protocol
IP	3 Network	Internet Protocol
LAN	2 Link 1 Physical	Local Area Network

1.1. Purpose

The purpose of the XMP is to provide a mechanism for preservation of message boundaries, typed messages, an optional "Extension Header" and a facility for message encapsulation.

1.2. Scope

This document specifies Exchange Message Protocol (XMP) - a mechanism that provides message level information and control when exchanging messages between SAXESS and its clients. Session encryption and message compression are not covered.

2. Overview

This document specifies a mechanism for providing message level information:

1. Message Length - preserves message boundaries when using stream-oriented transport protocols.
2. Message Type - provides message format multiplexing
3. Extension Header - provides a mechanism to add new pieces of information (e.g. message integrity, instrumentation)

XMP is used on the following SAXESS message flows:

1. Interactive (INT)
2. Unicast (UC)
3. Multicast (MC)

The XMP maximum message length is 3072 bytes.

3. XMP Header Details

3.1. XMP Base Header

The XMP base header consists of four bytes that are to be interpreted as three fields: BodyType, Extension Length and Body Length.

TABLE 2: XMP frame layout

Byte 1	Byte 2	Byte 3	Byte 4	Byte 5 - (if any)	Byte 5+ExtLen - (if any)
`BodyType`	`ExtLen`	`BodyLen1`	`BodyLen2`	*`Extension header ...`*	*`Body ...`*

TABLE 3: XMP header fields

Field	Description
`BodyType`	A type descriptor for the body part of the message
`ExtLen`	Length of the Extension Header in bytes A `0` (zero) indicates that no extension header is present.
`BodyLen`	The length of the body part of the message in bytes. BodyLen is represented as a two-byte integer in network byte order: BodyLen is formed by concatenating BodyLen1 and BodyLen2, with BodyLen1 being the most significant part of the length. `[C expression: BodyLen = BodyLen1 << 8 | BodyLen2]`

3.1.1. XMP Body Types

The table below specifies valid body types. All other type values are undefined and must not be used. Applications using this protocol should ignore any messages with unknown body types (possibly after having been logged for later analysis).

TABLE 4: Currently defined body types

Body Type	Value (hex)	The frame contains
XMP_Type_None	00	No data should be present (BodyLen=0)
XMP_Type_XTP_2	04	An XTP message used in XTP 2.20 and following versions.
XMP_Type_BinaryRecord (1)	05	A binary record
XMP_Type_BinaryTagged (1)	06	A message containing binary tagged fields
XMP_Type_KeyExchange (2)	07	A key exchange message (used for encrypted sessions)
XMP_Type_Encrypted (2)	81	An Encrypted frame
XMP_Type_Compressed (2)	82	A sequence of Compressed frames

NOTE

(1) The Binary Record and Binary Tagged types are currently for internal SAXESS use.

(2) As described below.

NOTE

All other Body Types are reserved. Messages containing a reserved Body Type should be ignored.

It is illegal to pass a ZERO (0) length body for any type except XMP_Type_None

3.2. XMP Extension Header

An XMP Extension Header, if present, consists of one or more tagged, variable length entries. Each entry type (identified by a tag) is defined in this protocol specification. The entry header consists of a **Tag** and a **Len** occupying one byte each. Valid tags are defined below. The length field defines the number of bytes of data that follows (excluding) the entry header. Applications using this protocol should be able to ignore an unknown extension header entry by skipping past the entry and continue with the next entry, if any.

TABLE 5: XMP Extension Header entry layout

Byte 1	Byte 2	Byte 3 - (if any)
Tag	Len	Data

3.2.1. Extension Header Entry Tags

The table below specifies valid entry tags for extension header entries. All other tag values are reserved and must not be used.

TABLE 6: Extension header entry tags

Tag Name	Value (hex)	Description
XMP_Tag_None	00	Entry should be skipped Data format: Any data will be skipped.
XMP_Tag_ClientTxnId	01	A client-specified transaction identification passed together with an XTP message. SAXESS then attaches this ID to the corresponding unicasts and multicasts messages. The ID value must be non-zero Data format: 4 byte unsigned integer.
XMP_Tag_ServiceTime	02	Service time (in SAXESS) for a transaction (in msecs) Data format: 4 byte unsigned integer.
XMP_Tag_KeepAlive	03	With this tag you can turn on/off keep alive handling. 0 = off, 1 = on Or, with a zero length datafield, you send an ordinary keep alive transaction. See also the discussion in the XTP specification Data format: No data or 1 byte unsigned integer.
XMP_Tag_Timestamp	04	If present, a microsecond resolution timestamp (UNIX time, microseconds). Data format: No data (a request for timestamp) or two 4 byte unsigned integers (8 bytes total). The first 4 bytes represents UNIX time, seconds since 1970-01-01 00:00:00 UTC. The following 4 bytes represents the number of microseconds within the second. Data format: Two 4 byte unsigned integers.
XMP_Tag_KeyId	05	A numeric ID for use in key exchange operations. Data format: 4 byte unsigned integer.

XMP_Tag_QueueAlert	06	Number of milliseconds that SAXESS has buffered messages (worst case) waiting to send to client. The minimum value is 500. Data format: 4 byte unsigned integer.
XMP_Tag_CompressionAlgorithm	07	Compression algorithm to be used (see table below) Data format: 1 byte unsigned integer.
XMP_Tag_EncryptionAlgorithm	08	Encryption algorithm to be used (see table below) Data format: 1 byte unsigned integer.
XMP_Tag_SessionState	09	Session State (see table below) Data format: 1 byte unsigned integer.
XMP_Tag_ProtocolVersion	0a	Protocol version (see table below) Data format: 1 byte unsigned integer.

NOTE

All multibyte integers are represented with the most significant byte first ("network byte order").

3.2.1.1. Compression Algorithms

A compression algorithm may only be specified (once) before application logon (i. e. in session state = Offline).

SAXESS supports compression of interactive replies, unicast and multicast flows. The frames are compressed individually, so the maximum size of a decompressed frame is equal to XMP maximum message length.

SAXESS may choose to send messages that have not been compressed. Consequently a member application must interpret each message and decompress based on the presence of an XMP header indicating that its body is compressed.

TABLE 7: Compression Algorithms

Algorithm Name	Value (hex)	Description
XMP_CA_None	00	No compression is performed
XMP_CA_LZ1	01	Compression is performed using LZ1

NOTE

LZ1 is an XTP optimized LZ77 variant

3.2.1.2. Encryption Algorithms

The encryption algorithm must be specified in the extension header of the key exchange request. It may not be used otherwise.

The algorithm used for key exchange is also used to encrypt messages during the session.

TABLE 8: Encryption Algorithms

Algorithm Name	Value (hex)	Description
XMP_EA_None	00	No encryption is performed
XMP_EA_RC4	01	Encryption is performed using RC4
XMP_EA_DES	02	Encryption is performed using DES-CBC

3.2.1.3. Session States

For INT sessions, the session state can be set by the member application to indicate its status towards the SAXESS system. SAXESS monitors any state changes sent by the member application and forwards them to the SAXESS system supervision facility.

SAXESS does not set the session state for INT sessions. If the client does not set the session state, it will default to XMP_SS_None.

For UC and MC sessions, the SAXESS system returns session state changes. An UC / MC session is initially in Offline state. When the member application logs on, the state is changed either to Catchup or Online depending on if previously generated data must be sent from the SAXESS system.

SAXESS sends state changes on UC / MC sessions. The member application may not set the session state.

TABLE 9: Session States

Session State	Value (hex)	Description
XMP_SS_None	00	The session state is undefined
XMP_SS_Offline	01	No application logon has been performed
XMP_SS_Catchup	02	The application using this session is performing catchup (recovery) of data from SAXESS
XMP_SS_Online	03	The application using this session is online (catchup has completed)

3.2.1.4. Protocol Versions

TABLE 10: Protocol Versions

Protocol Version	Value (hex)	Description
XMP_PV_None	00	Undefined version
XMP_PV_Current	01	Version 1

3.2.1.5. Extension Header Entry Examples

TABLE 11: Example Extension Header entries (see TABLE 6)

Byte stream (hex)	Description
01 04 12 34 56 78	**ClientTxnId = 0x12345678**
02 04 00 00 00 1e	**ServiceTime = 30 milliseconds**
03 00	**KeepAlive**
03 01 01	**KeepAlive = ON**
05 04 12 13 45 78	**KeyId = 0x12345678**
06 04 00 00 02 00	**QueueAlert = 512 milliseconds**
07 01 01	**CompressionAlgorithm = LZ1**
08 01 02	**EncryptionAlgorithm = DES**
09 01 02	**SessionState = Online**
0a 01 01	**ProtocolVersion = 1**

3.2.2. Extension Header Examples

3.2.2.1. Round Trip Instrumentation

The **ClientTxnId** can be passed from a client application to SAXESS along with any of the defined XTP request messages. The SAXESS system then passes the same entry back in the **Interactive reply** as well as in **Unicast** messages generated by SAXESS as a direct result of the request.

If enabled for a specific session, SAXESS calculates and passes a millisecond resolution **ServiceTime** for each **Interactive Reply** and related **Unicast** messages.

The example below depicts an **InsertOrder** request generating a **Unicast** message. Version 1 enables **ServiceTime** by default if a **ClientTxnId** is passed in the request.

TABLE 12: OrderInsert Request Message (Client->SAXESS on INT)

Byte stream (hex)	Size	Description
04 06 00 9c	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 6 bytes [06] BodyLen = 164 bytes [00 a4]
01 04 12 34 56 78	6	ClientTxnId = 0x12345678
.. ..	164	XTP **OrderInsert_5** message

TABLE 13: OrderInsert Reply Message (SAXESS->Client on INT)

Byte stream (hex)	Size	Description
04 0c 00 2a	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 12 bytes [0c] BodyLen = 42 bytes [00 2a]
01 04 12 34 56 78	6	ClientTxnId = 0x12345678
02 04 00 00 00 0a	6	ServiceTime = 10 ms
.. ..	42	XTP **OrderInsert_5** reply

TABLE 14: OrderInsert Unicast Message (SAXESS->Client UC)

Byte stream (hex)	Size	Description
04 0c 00 fa	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 12 bytes [0c] BodyLen = 250 bytes [00 fa]
01 04 12 34 56 78	6	ClientTxnId = 0x12345678
02 04 00 00 00 14	6	ServiceTime = 20 ms
.. ..	250	XTP **UC_OrderInsert_9** message

3.2.2.2. Interactive KeepAlive with Data

TABLE 15: KeepAlive Request Message (Client->SAXESS on INT)

Byte stream (hex)	Size	Description
00 03 00 00	4	XMP Base Header BodyType = None [00] ExtHdrLen = 3 bytes [03] BodyLen = 0 bytes [00 00]
03 01 01	3	KeepAlive = ON [01]

NOTE

The client requests that KeepAlive should be enabled.

Alternatively the client could request to turn KeepAlive checking OFF [00]. SAXESS is normally configured to accept such requests in a test system. KeepAlive is required in a production system.

TABLE 16: KeepAlive Reply Message (SAXESS->Client on INT)

Byte stream (hex)	Size	Description

`00 03 00 00`	4	`XMP Base Header`
		`  BodyType  = None      [00]`
		`  ExtHdrLen =   3 bytes [03]`
		`  BodyLen   =   0 bytes [00 00]`

`03 01 01`	3	`KeepAlive = ON [01]`

NOTE

SAXESS acknowledges that KeepAlive is enabled.

SAXESS would return KeepAlive = OFF if KeepAlive is disallowed for some reason. Conversely, if the client tries to disable KeepAlive, SAXESS may reject the request by returning KeepAlive = ON.

3.2.2.3. Interactive Dataless KeepAlive

The Interactive Dataless KeepAlive request is used to to verify the TCP connection when the application is not transmitting data. The maximum time between two XMP frames is specified in the XTP Logon transaction by the Session Alive Timeout.

TABLE 17: KeepAlive Request Message (Client->SAXESS on INT)

Byte stream (hex)	Size	Description

`00 02 00 00`	4	`XMP Base Header`
		`  BodyType  = None      [00]`
		`  ExtHdrLen =   2 bytes [02]`
		`  BodyLen   =   0 bytes [00 00]`

`03 00`	2	`KeepAlive Request`

NOTE

The Client does not pass any data.

TABLE 18: KeepAlive Reply Message (SAXESS->Client on INT)

Byte stream (hex)	Size	Description
00 02 00 00	4	XMP Base Header BodyType = None [00] ExtHdrLen = 2 bytes [02] BodyLen = 0 bytes [00 00]
03 00	2	KeepAlive Reply

NOTE

(1) SAXESS acknowledges the KeepAlive.

3.2.2.4. Unicast / Multicast KeepAlive

KeepAlive is always enabled on Unicast and Sessionbased Multicast sessions.

The KeepAlive message is dataless.

TABLE 19: KeepAlive Message (SAXESS->Client on UC / MC)

Byte stream (hex)	Size	Description
00 02 00 00	4	XMP Base Header BodyType = None [00] ExtHdrLen = 2 bytes [02] BodyLen = 0 bytes [00 00]
03 00	2	KeepAlive

NOTE

SAXESS sends KeepAlive messages spontaneously (by default) on the UC and MC flows. The client should NOT respond to these messages.

3.2.2.5. QueueAlert Configuration Request

The QueueAlert can be configured for UC and MC sessions. A threshold value can be set either in a separate XMP frame before logon (see example below) or in the extension header of the logon request. Any attempt to (re)set the threshold value after logon will be ignored.

TABLE 20: QueueAlert Request Message (Client->SAXESS on UC)

Byte stream (hex)	Size	Description
00 06 00 00	4	XMP Base Header BodyType = None [00] ExtHdrLen = 6 bytes [06] BodyLen = 0 bytes [00 00]
06 04 00 00 02 00	6	QueueAlert = 512 [00 00 02 00]

NOTE

Request QueueAlert threshold to be set to 512.

TABLE 21: QueueAlert Reply Message (SAXESS->Client on UC)

Byte stream (hex)	Size	Description

`00 06 00 00`	4	`XMP Base Header`
		`BodyType = None [00]`
		`ExtHdrLen = 6 bytes [06]`
		`BodyLen = 0 bytes [00 00]`

`06 04 00 00 02 00`	6	`QueueAlert = 512 [00 00 02 00]`

NOTE

SAXESS confirms that QueueAlert threshold has been reset (to 512). A QueueAlert will be issued whenever SAXESS must buffer the outgoing flow and the resulting delay exceeds the threshold value (512 milliseconds). The data part of a QueueAlert will then contain the actual delay in milliseconds.

The first QueueAlert will contain a value greater than 512. A QueueAlert with a value less than 512 will be sent, when the delay drops below 512.

3.2.3. Timestamp Request

A data-less `XMP_Tag_Timestamp` entry that is passed (Client->SAXESS) on an INT, UC or MC session is interpreted as a request for the current SAXESS system time.

TABLE 22: Timestamp Request Message (Client->SAXESS on INT)

Byte stream (hex)	Size	Description

`00 02 00 00`	4	`XMP Base Header`
		`BodyType = None [00]`
		`ExtHdrLen = 2 bytes [02]`
		`BodyLen = 0 bytes [00 00]`

`04 00`	2	`Timestamp Request`

NOTE

A timestamp request can be issued at all times, but will be ignored after logon on UC and MC flows.

TABLE 23: Timestamp Reply Message (SAXESS->Client on INT)

Byte stream (hex)	Size	Description
00 0a 00 00	4	XMP Base Header BodyType = None [00] ExtHdrLen = 10 bytes [0a] BodyLen = 0 bytes [00 00]
04 08 38 31 a3 3d 00 00 00 00	10	Timestamp = 1999-11-16 18:32:29.000000 UTC

4. Encapsulation

Complete frames can be encapsulated in another frame. This feature can be used to implement layered functions like compression and encryption.

A compressed body may contain multiple encapsulated frames, while an encrypted body contains exactly one frame. The reason for this restriction is to allow unambiguous interpretation of any pad bytes added for block ciphers. If there are any bytes following the encapsulated frame (after decryption), they should be discarded. Otherwise, it would be impossible to distinguish pad bytes from the start of another frame.

The encapsulation frames are identified by Body Type as described in table 5 above.

Except for Key Exchange (see below), SAXESS will not emit any extension header entries on encapsulated frame types. Conversely, except for Key Exchange, SAXESS will ignore extension header entries on incoming encapsulation frames.

4.1. Encapsulation Examples

4.1.1. Example - An Encrypted XTP message

TABLE 24: Encrypted XTP message

Byte stream (hex)	Size	Description
81 00 00 9e	4	XMP Base Header BodyType = Encrypted [81] ExtHdrLen = 0 bytes [00] BodyLen = 168 bytes [00 a8]
		The entries below are encrypted
04 00 00 9a	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 0 bytes [00] BodyLen = 164 bytes [00 a4]
.. ..	164	XTP **OrderInsert_5** request

4.1.2. Example - A Compressed Sequence of Frames

TABLE 25: Compressed XTP message

Byte stream (hex)	Size	Description
82 00 xx yy	4	XMP Base Header BodyType = Compressed [82] ExtHdrLen = 0 bytes [00] BodyLen = xxx bytes [xx yy]
		The entries below are compressed (shown decompressed)
04 00 00 fa	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 0 bytes [00] BodyLen = 250 bytes [00 fa]
.. ..	250	XTP **UC_OrderInsert_9** message
04 00 00 fa	4	XMP Base Header BodyType = XTP_2 [04] ExtHdrLen = 0 bytes [00] BodyLen = 250 bytes [00 fa]
.. ..	250	XTP **UC_OrderInsert_9** message

5. Key Exchange (for Session Encryption)

Key Exchange is performed by a request / reply message scheme to enable encryption for a session. The Key Exchange protocol assumes that the parties involved share a 128 bit secret key identified by key ID. The secret key is only used to encrypt / decrypt the key exchange messages. The session key returned in the second key exchange message is then used to encrypt/decrypt subsequent messages.

The timestamp passed in the Key Exchange request must be within 5 seconds accurate compared to SAXESS system time. This restriction is posed to prevent replay attacks. SAXESS current system time can be retrieved using a XMP_Tag_Timestamp request (se example above).

The transport connection will be closed without any error indication if a Key Exchange request fails (any error indication could provide useful information to an attacker).

TABLE 26: Key Exchange message

Field	Size (bytes)	Format	Description
Opcode	1	int	1=Request 2=Reply
Nonce	7	byte	Random bytes to increase entropy
KeyId	4	int	Key identification
Timestamp	4	int	Unix time
Key	16	byte	Random session key

TABLE 27: Key Exchange Request (Client->SAXESS)

Byte stream (hex)	Size	Description
81 09 00 28	4	XMP Base Header BodyType = Encrypted [81] ExtHdrLen = 9 bytes [09] BodyLen = 40 bytes [00 28]
05 04 00 00 00 23	6	KeyId [05] = 35 [00 00 00 23]
08 01 02	3	Encryption [08] = DES [02]
		All entries below are encrypted
07 00 00 20	4	XMP Base Header BodyType = KeyExchange [07] ExtHdrLen = 0 bytes [00] BodyLen = 32 bytes [00 20]
01	1	Opcode = Request [01]
.. ..	7	Nonce data (7 bytes)

00 00 00 23	4	KeyId = 35
38 31 a3 3d	4	Timestamp = 1999-11-16 18:32:29 UTC
.. ..	16	Random Session Key (1)
.. ..	4	Pad bytes (DES block size is 8 bytes)

NOTE

The contents of session key is ignored

The Nonce field should be filled with random data to increase entropy

DES plain text size must be a multiple of 8 bytes.

TABLE 28: Key Exchange Reply (SAXESS->Client)

Byte stream (hex)	Size	Description
81 09 00 28	4	XMP Base Header BodyType = Encrypted [81] ExtHdrLen = 9 bytes [09] BodyLen = 40 bytes [00 28]
05 04 00 00 00 23	6	KeyId [05] = 35 [00 00 00 23]
08 01 02	3	Encryption [08] = DES [02]
		All entries below are encrypted
07 00 00 20	4	XMP Base Header BodyType = KeyExchange [07] ExtHdrLen = 0 bytes [00] BodyLen = 32 bytes [00 20]
02	1	Opcode = Reply [02]
.. ..	7	Nonce data (7 bytes)
00 00 00 23	4	KeyId = 35
38 31 a3 3d	4	Timestamp = 1999-11-16 18:32:29 UTC
.. ..	16	Random Session Key
.. ..	4	Pad bytes (DES block size is 8 bytes)